

COMPUTER VISION • PHONE CAMERA LAB 4

From photographs to 3D

Calibration, correspondences, homography, and stereo

Your task: turn your phone into a two-view measurement instrument. Calibrate one camera, capture your own image pairs, estimate their geometry, and reconstruct a small set of scene points. Use **one camera moved between viewpoints** as the default, even when your phone has several cameras.

Format: one assignment across two suggested 90-minute sessions. Work in pairs. Complete A–E using OpenCV routines; you are not required to implement SIFT, RANSAC, calibration, or a pose solver from scratch. Collect the images before class where possible, and finish the report afterwards. Dense depth is an extension, not a prerequisite for completing this introductory lab.

Meeting	Suggested allocation	Main result
Session 1	A 40 min; B 30 min; C 20 min	Calibration, planar alignment, and a controlled stereo pair
Session 2	D 35 min; E 40 min; review 15 min	Epipolar lines, relative pose, disparity measurements, and sparse 3D

Equipment and capture rules

Bring a phone, laptop, ruler, stable support, a flat printed checkerboard, and textured objects at several depths. A board with **9 × 6 inner corners** contains **10 × 7 squares**. Measure the actual printed square size and record it in meters; do not assume the printer preserved its intended size. Mount the board flat.

Choose one rear camera and keep lens, zoom, focus setting, resolution, and orientation fixed throughout calibration and the stereo experiment. Use good lighting, no flash, no synthetic blur, and no moving scene objects. Record unavoidable automatic processing. Preserve image dimensions and metadata when transferring files. If focus cannot be locked, minimize refocusing and report it as a calibration limitation.

Single-camera stereo: take the left image, translate the phone, wait until it is still, and take the right image. Two camera centers are needed; two physical cameras are not. The scene must stay unchanged between these exposures. Simultaneous capture matters when the scene moves, but is not required for this static-scene experiment.

Multiple-camera option: separately calibrate each camera and account for its own distortion and intrinsics. Do not subtract pixel coordinates from unrectified main/ultra-wide images. Do not assume the stock app exports synchronized individual views. All students can use the single-camera route for full credit.

Connection to the lecture material

Calibration and distortion follow lec4-1.pdf, pp. 8–11, and lect4-2.pdf, pp. 13–14 and 19–27. Matching and robust fitting connect to lec3.pdf, pp. 4–8, 61–68, and 102–116. Epipolar geometry and rectification connect to 03-epipolar-geometry.pdf, pp. 2–11, and lect4-2.pdf, pp. 40–49. Reconstruction follows the sequence in lect4-3.pdf, pp. 5–7 and 25–31.

Notation: use $X_R = RX_L + t$ and $x_R^T F x_L = 0$, consistent with the view ordering in lect4-3.pdf, pp. 21–22, and OpenCV. The CS231A notes use a different ordering in their epipolar equations; do not combine formulas across conventions without translating the definitions.

SESSION 1 • CALIBRATE THE EXPORTED IMAGES

A. Estimate camera intrinsics and distortion

Capture a useful calibration set

Take **18–25 photographs** of the checkerboard using the camera configuration chosen for the stereo scene. Keep the entire board visible. Vary its tilt around both axes and its location across the image, including near the edges; vary distance moderately while keeping corners sharp. Do not submit many nearly identical frontal views. Keep three varied images aside for validation.

Detect inner corners using `cv.findChessboardCornersSB`, or `cv.findChessboardCorners` followed by `cv.cornerSubPix`. Draw the detected corners and inspect their ordering. Record failed detections. See the OpenCV calibration tutorial for implementation guidance. [W1]

For square size s , construct board points $(is, js, 0)$ using the detected corner ordering. All board points are coplanar, but the calibration uses multiple board poses, as in the Zhang-method material in `lect4-2.pdf`, pp. 19–27.

Calibrate, check, and save

Run `cv.calibrateCamera` on the training views and save the resulting matrix K , distortion coefficients, image size, and per-view poses. Report f_x, f_y, c_x, c_y in **pixels**, not millimeters. Include the board square size and the exact camera configuration in the saved metadata.

Reproject the board corners with `cv.projectPoints`. Compute and report the pointwise reprojection RMS, using the same definition for every set:

$$e_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{i=1}^N \|p_i - \hat{p}_i\|_2^2}.$$

Here N counts individual observed corners, not images. Display per-view errors and one observed-versus-reprojected overlay. For each held-out image, keep K and distortion fixed, estimate only the board pose with `cv.solvePnP`, and compute its error. These are held-out intrinsic-model checks, not independent measurements of physical depth accuracy.

Undistort a checkerboard image and compare line straightness and image boundaries. A small change is acceptable: the **exported image pipeline** may already include lens correction.

Preserve the coordinate system

Do not use a full-resolution calibration matrix unchanged on resized images. For a resize with horizontal and vertical factors s_x, s_y :

$$K_{\text{new}} = \text{diag}(s_x, s_y, 1)K.$$

A crop also shifts the principal point by the crop offset. Use one processing resolution throughout calibration and reconstruction; preserve orientation. Start near 1280 pixels on the long side for classroom computation. [W2]

Checkpoint A: show the instructor your detected corners, K , distortion coefficients, image dimensions, and error plot. Explain why a low fitting error alone does not prove a correct physical scale or guarantee accuracy on other phone settings. Recapture blurry or poorly distributed views before continuing.

SESSION 1 • MATCHING AND CONTROLLED VIEWPOINTS

B. Planar alignment with a homography

Photograph a textured flat poster or book cover from two moderately different viewpoints with substantial overlap. Use the undistorted processing images from your calibrated pipeline; this pair is separate from the multi-depth stereo pair in C.

Detect and describe SIFT features, then match descriptors with a two-nearest-neighbor matcher and a ratio test, initially 0.75. Alternatively use ORB with Hamming distance; do not use the same distance metric blindly for both descriptor types. Display candidate correspondences before robust fitting. [W3]

Estimate a homography with `cv.findHomography(..., cv.RANSAC, 3.0)` and display only its inliers. The 3-pixel threshold is a starting value at your processing resolution; record any change. Warp one image with `cv.warpPerspective`, then overlay the common plane or create a simple two-image mosaic. This is a planar alignment experiment, not a complete panorama system.

Record the number of keypoints, ratio-test matches, homography inliers, inlier fraction, and median inlier transfer error. A homography needs at least four correspondences in a nondegenerate arrangement; use many spatially distributed matches in practice. The mapping for this task is $x_2 \sim Hx_1$.

Question: where does alignment fail when a foreground object lies in front of the poster? Explain why one homography can describe a plane but cannot generally align all depths after camera translation. It is not interchangeable with a fundamental matrix.

C. Capture a stereo pair with one camera

Create a static scene with textured objects at **three noticeably different depths**, for example around 0.6 m, 1 m, and 1.5 m. Adapt the arrangement so everything is sufficiently sharp and both views retain substantial overlap. Avoid blank walls, shiny surfaces, and a single dominant plane as your only source of features.

Support the phone at a marked left position. Take `left.jpg`. Move it **sideways by approximately 5–10 cm**, keeping height and orientation as constant as possible. Take `right.jpg` after the phone is still. Do not simply rotate the phone about its lens and do not deliberately point both views inward. Record the measured displacement and its approximate uncertainty, for example the ruler's reading precision plus placement error.

Measure the displacement of the chosen camera center, not a changing phone corner. For approximately pure translation with unchanged orientation, a fixed mark on the phone undergoes the same displacement as the lens. Record this as the measured baseline B in meters. It is a practical scale measurement, not an exact factory calibration.

Include two visible endpoints on a rigid object whose separation you can measure independently, such as marked points 15–25 cm apart. This length will be used to validate reconstruction; **do not use it to set the reconstruction scale** if you plan to report it as a test.

Before leaving Session 1: check that features match on more than one depth layer, not only on the checkerboard or wall. Keep the scene available until the first geometry checks succeed. Save the originals, camera log, baseline measurement, and the independently measured test length.

SESSION 2 • EPIPOLAR GEOMETRY AND SCALE

D. Estimate the relationship between the views

Fundamental matrix: a point predicts a line

Undistort the two images from C using the same K as the output camera matrix and without cropping. Match SIFT or ORB features as in B. Estimate F with RANSAC using `cv.findFundamentalMat`; start with a 1–2 pixel threshold at your processing resolution. Use at least 20 well-spread, visually checked matches when possible, including more than one depth layer. [W4]

Use homogeneous pixel coordinates $x_L = (u_L, v_L, 1)^T$ and $x_R = (u_R, v_R, 1)^T$. Draw several corresponding points and their epipolar lines:

$$x_R^T F x_L = 0, \quad \ell_R = F x_L, \quad \ell_L = F^T x_R.$$

Compute the point-to-line distance in pixels, rather than interpreting the scale-dependent algebraic residual as a distance:

$$e_R = \frac{|x_R^T F x_L|}{\sqrt{(F x_L)_1^2 + (F x_L)_2^2}}.$$

Report the median and 90th percentile for inliers, and examine rejected matches. Also inspect at least five manually checked correspondences that were not used to fit F . If the matrix fits one plane but not the other objects, improve the match distribution or recapture the scene.

Essential matrix: separate intrinsics from pose

For calibrated, distortion-corrected coordinates, the relationship is

$$E = K_R^T F K_L, \quad \tilde{x}_R^T E \tilde{x}_L = 0, \quad \tilde{x} = K^{-1} x.$$

For the single-camera route, $K_L = K_R = K$. Compute $K^T F K$ for comparison. For pose recovery, estimate a constrained essential matrix directly with `cv.findEssentialMat` on the undistorted pixel correspondences and K , then call `cv.recoverPose` with those same points, K , and the inlier mask. Do not pass normalized points together with the original pixel-space K . [W2]

Report the rotation and the number of correspondences passing the positive-depth check. `recoverPose` returns a **translation direction** $\hat{t}$ with unit length, not a displacement in meters. When the measured baseline is credible, set

$$t = B \hat{t}.$$

Retain this same R, t and coordinate convention for all subsequent rectification and triangulation. The reconstructed scale will inherit baseline measurement error. Without a measured baseline or another justified scale constraint, continue in **arbitrary units** and do not label your result as metric depth.

Checkpoint D: explain what H , F , and E do differently. State exactly where metric scale entered your pipeline. Merely placing a known-size checkerboard somewhere in a photograph does not automatically supply scale to a feature-only reconstruction.

Recovery option: if feature-based pose estimation is unstable, use the fixed-board PnP route described in the instructor notes. Keep the failed result and identify which pose source generated your final output; do not mix outputs from incompatible poses.

SESSION 2 • FROM CORRESPONDENCES TO 3D

E. Rectify, measure disparity, and triangulate

Rectify the calibrated pair

Run `cv.stereoRectify` with the original calibrated intrinsics, distortion coefficients, and the R, t from D. Use `cv.CALIB_ZERO_DISPARITY` for this exercise. It returns rectification rotations, new projection matrices, and the disparity-to-3D matrix Q . Use `cv.initUndistortRectifyMap` and `cv.remap` on the original processing images. Do not apply the original distortion correction twice. [W2]

Draw horizontal guide lines on the rectified pair. Transform your matched points into the rectified coordinate systems and measure $|v_L - v_R|$. Report its median and 90th percentile. A practical target is approximately 1–2 pixels at the processing resolution, not a guarantee or an automatic pass criterion. Fix conspicuous vertical mismatch before interpreting disparity.

Measure a small, trustworthy disparity set

Choose at least **six corresponding points**, with two on each depth layer. Include the two independently measured endpoints from C. Inspect them manually in the rectified images and record (u_L, v_L, u_R, v_R) , disparity, and estimated depth.

For the approximately horizontal-baseline layout used here, with equal rectified principal points and a consistent left/right order,

$$d = u_L - u_R, \quad Z = \frac{f_{x,\text{rect}} B}{d}.$$

Use focal length in **rectified pixels**, from the rectified projection matrix, and the baseline in meters. Reject zero or invalid disparity; do not use $|d|$ to conceal a wrong view order. Near points should generally have larger disparity. Without equal rectified principal points, their horizontal offset must also enter the depth formula. Do not apply this simplified formula directly to the unrectified photographs.

Discuss how a one-pixel error affects near and far measurements. The inverse relationship between disparity and depth appears in `lect4-2.pdf`, pp. 41 and 44; its p. 45 illustrates the growth of depth error with distance.

Build and validate a sparse point cloud

Triangulate the inlier correspondences using `cv.triangulatePoints`. One consistent route uses the undistorted pixel points from D with

$$P_L = K[I \mid 0], \quad P_R = K[R \mid t].$$

Divide the returned homogeneous coordinates by their fourth component. Reject non-finite points, points behind either camera, and points with large reprojection error. Display the retained points in a 3D scatter plot with labeled axes and units. Use equal axis scaling so the plot does not exaggerate shape. A sparse point cloud is sufficient; a watertight model is not expected.

For points selected in rectified images, use the rectified projection matrices instead; never mix rectified points with unrectified matrices. Triangulate the two marked endpoints and compare their 3D separation with the ruler measurement that was not used to set scale. Report absolute error and percentage error. Explain possible causes, including baseline error, calibration, matching, and small triangulation angles. Treat a large error as a result to diagnose, not a reason to rescale against the test length and present that as independent validation.

SUBMISSION • LAB 2

Evidence, extensions, and assessment

Submit Lab2_GroupID: original images, capture/measurement logs, saved calibration and pose, runnable code, exported figures, and a **report of no more than five pages**. Include the actual image size, camera configuration, processing choices, all measurement units, and any fallback data or method used.

Core section	Required evidence	Points
A. Calibration	Corner overlays, K , distortion, per-view/held-out errors, before/after undistortion	25
B. Homography	Candidate/inlier matches, planar alignment, match counts, transfer error, limitation	15
C. Capture	Own multi-depth pair, reproducible settings, measured baseline and independent test length	15
D. Epipolar geometry	F , E , epipolar lines, point-to-line errors, pose, explicit source of scale	20
E. Reconstruction	Rectification check, six-point disparity table, filtered cloud, independent length error	25

Full marks do not require a new phone, a dense depth map, or a visually perfect reconstruction. They require correctly controlled data, a valid pipeline, meaningful checks, and a clear explanation of limitations. When metric reconstruction remains unreliable after a documented retry, report relative geometry and analyze the failed metric result rather than inventing accuracy.

Optional extensions — choose one

Dense stereo and a depth image. Apply `cv.StereoSGBM_create` to the rectified grayscale pair. Start with an odd block size of 5 or 7 and a disparity search range that is a multiple of 16 and covers the observed disparities. Convert its fixed-point result to floating-point pixels by dividing by 16. Mask invalid matches, rectification borders, and visibly unreliable regions. Use the matching Q with `cv.reprojectImageTo3D`; do not reuse it after changing resolution or crop. Show the disparity/depth image and identify an occlusion or textureless failure. [W2, W5]

Baseline comparison. Repeat C–E with a different measured baseline. Compare correspondence reliability and reconstruction error. Relate the trade-off to `lect4-3.pdf`, p. 49.

More views or phone stereo features. Capture 10–20 overlapping views of a static object for guided SfM (`lect4-3.pdf`, pp. 6–7, 34). Alternatively inspect a phone stereo/spatial export: verify that it contains two views and identify its actual metadata. Neither replaces the core experiment.

Final discussion

When can one translated camera replace simultaneous stereo? Contrast what exposure, focus, and viewpoint sequences add. What would a learned single-image depth model infer from prior training rather than measure from your baseline?

Implementation references

[W1] OpenCV: [Camera calibration](#). [W2] OpenCV: [Calibration and 3D reconstruction API](#). [W3] OpenCV: [Matching and homography](#). [W4] OpenCV: [Epipolar geometry](#). [W5] OpenCV: [Stereo depth](#).